Introduction to Google Agent Development Kit (ADK)

1. What is Google ADK?

Google Agent Development Kit (ADK) is an open-source framework for building, testing, evaluating, and deploying AI agents. An AI agent is more than a chatbot: it receives a goal, interprets instructions, may call tools, and may coordinate with other agents to complete a task.

ADK supports development in several languages. This laboratory uses the Python version and the ADK Web Visual Builder. The Visual Builder is an experimental feature available through the ADK web interface that lets students create and connect agents visually.

Official references:

- Google ADK overview
- Python quickstart
- ADK Visual Builder

2. Why use an agent framework?

A normal program follows rules written directly in code. A language model can interpret natural-language requests, but it should not be trusted to invent factual records or perform unrestricted actions. ADK combines both approaches:

- the language model interprets the user's intention;
- instructions define the agent's responsibility and boundaries;
- tools perform controlled, deterministic operations;
- specialist agents divide a large task into smaller responsibilities; and
- workflow agents control the order or pattern of execution.

This separation makes an application easier to understand, test, and improve.

3. Core ADK concepts

3.1 LLM agent

An `LlmAgent` is an agent controlled by a language model. It normally has:

- a unique name;
- a model;
- instructions;
- an optional description;
- optional tools; and
- optional sub-agents.

For example, a billing agent may interpret a bill request, obtain an account alias, call a bill-

summary tool, and explain the returned result.

An effective LLM-agent instruction should state:

1. its responsibility;
2. the information it needs;
3. the information it must not request or handle;
4. the tools it owns;
5. what to do when information is missing;
6. its expected response style; and
7. its authority and side-effect boundaries.

ADK requires agent names to be valid and useful identifiers. In multi-agent applications, names are important because agents and routing instructions refer to them. See ADK LLM agents.

3.2 Root agent

The root agent is the application's main entry point. It receives the user's message first.

In a multi-agent system, the root agent can act as a coordinator. It should:

- understand the type of request;
- select one suitable child agent or workflow;
- transfer the request; and
- avoid doing specialist work itself.

Keeping specialist tools away from the root reduces accidental tool calls and makes event traces easier to interpret.

3.3 Sub-agent

A sub-agent is a child of another agent. A root agent may have several specialists, such as:

- an intake agent;
- a classification agent;
- an outage agent;
- a billing agent; and
- a response agent.

An LLM parent can transfer a request to an appropriate child. The event history may display a `transfer_to_agent` action during this process.

3.4 Tools

A tool gives an agent a controlled capability. A Python Function tool wraps a Python function and exposes its parameters and result to the agent.

Examples include:

```
classify_issue(issue_description, area="")
check_outage_status(area)
get_bill_summary(account_alias)
```

Tools should have narrow responsibilities and structured results. A read-only lookup tool should not modify a file. A write tool should validate its inputs and enforce authorization rather than depending only on an LLM instruction.

Important rule:

> Register a tool on the agent that owns the responsibility—not on the root or an unrelated specialist.

3.5 Built-in tools

ADK may provide built-in tools in addition to custom Python functions. In this project, `get_user_choice` displays controlled options to collect explicit consent.

A built-in choice tool is preferable to treating arbitrary user text as permission for a write operation.

3.6 Sessions and events

A session represents one conversation. During development, start a new session for each independent test so that earlier messages do not influence the next test.

ADK Web displays events generated during execution. An event trace can show:

- which agent ran;
- whether control was transferred;
- which tool was called;
- the tool input;
- the tool output; and
- where an error occurred.

The Web interface supports session management, event inspection, and state inspection. See ADK Web interface.

4. Workflow agents

Workflow agents are deterministic controllers. Unlike an `LlmAgent`, a workflow agent does not use a model to decide its execution pattern.

4.1 SequentialAgent

A `SequentialAgent` executes its children one after another in the order in which they were added.

Use it when later stages depend on earlier stages:

```
Understand request
→ classify issue
→ retrieve record
```

```
→ explain result
```

The order of children is therefore essential. See ADK Sequential workflow.

4.2 ParallelAgent

A `ParallelAgent` executes its children concurrently. Use it only when the branches are independent and all required inputs are already available.

Example:

```
              ┌→ outage lookup
User request ─┤
              └→ bill lookup
```

A parallel workflow should not be used when one child must wait for the output of another. See ADK Parallel workflow.

4.3 LoopAgent

A `LoopAgent` repeats its child workflow until a stopping condition or maximum iteration count is reached.

In a classroom prototype, a small maximum iteration count prevents repeated questioning or an accidental infinite loop.

4.4 Agent parent limitation

In the Visual Builder used for this laboratory, the **Sub Agents +** menu creates a new `LlmAgent`, `SequentialAgent`, `ParallelAgent`, or `LoopAgent`. It does not provide a command for selecting an existing agent.

ADK also assigns a parent to each agent instance. One agent instance cannot simultaneously belong to several parents. Therefore, workflow-specific copies need unique names, for example:

```
seq_triage_agent
par_triage_agent
```

These agents may use the same instruction pattern and Python tool, but they are separate agent instances.

5. Model versus tool responsibilities

The language model is appropriate for:

- understanding natural-language requests;
- deciding which known intent applies;
- asking a concise follow-up question; and
- explaining structured results.

A deterministic Python tool is appropriate for:

- reading a CSV file;
- applying exact matching rules;

- checking for a duplicate;
- allocating an identifier; and
- appending a validated row.

Do not ask an LLM to calculate or invent a value already stored in an authoritative local record.

6. Read-only operations and side effects

A side effect changes something outside the current response. Examples include writing a CSV row, sending a message, or processing a payment.

This laboratory distinguishes between:

- **read-only tools**, which retrieve or classify information; and
- **write tools**, which modify `support_cases.csv`.

A safe write flow should be:

```
Present proposed data
→ obtain explicit choice
→ check for duplicate
→ validate again
→ perform one write
→ report the tool's result
```

7. Development workflow

A reliable ADK development cycle is:

1. define the application scope;
2. design tool contracts;
3. design specialist responsibilities;
4. create and test Python functions;
5. create the root and specialists;
6. attach every tool to its owning agent;
7. test each specialist separately;
8. create and test workflows;
9. inspect events and side effects;
10. correct one problem at a time
11. retest in a new session.